

MUNICTECH JUDGE SCORING EVIDENCE MAP

AI Action Path Assurance by HAIEC

A one-page guide to the strongest evidence for each MunichTech judging criterion.

RECOMMENDED CATEGORY

Best Industry & Enterprise Use Case

Why this category fits:

AI Action Path Assurance is an enterprise-focused assurance product with a live SaaS implementation, a real multi-tenant AI test application, source-backed technical evidence, release artifacts, and a deployment model designed for security, platform, assurance, and governance teams.

1. PROBLEM RELEVANCE & IMPACT

Judge question:

Does this solve a real, important problem with meaningful impact?

Our answer:

AI agents are moving from generating text to operating tools, APIs, databases, cloud services, and enterprise workflows. Most organizations can inspect permissions or tool inventories, but still struggle to prove the full path from AI exposure to a consequential system action.

Proof:

- KestrelVoice is a functional multi-tenant AI SaaS application used as a real-world test application.
- HAIEC confirmed 44 consequential AI action paths in the evaluated source.
- The hero path reaches an ORDER RECORD WRITE through a source-backed chain.
- REVIEW converts that technical capability into an explicit release decision rather than a raw scanner finding.

Why it matters at scale:

The same assurance question appears wherever AI can change records, transfer money, modify infrastructure, send messages, alter configurations, invoke production tools, or act across tenant boundaries.

2. TECHNICAL EXCELLENCE & FEASIBILITY

Judge question:

Is the implementation technically strong, credible, and deployable?

Our answer:

HAIEC uses a deterministic assurance stack rather than asking an AI model to judge another AI system.

How it works:

Exact source snapshot

- AI-facing capability identification
- model exposure / registration qualification
- dispatch continuity
- implementation and handler binding
- consequential operation
- arguments, context, controls, services, resources, persistence
- five-plane evidence reconciliation
- bounded assurance decision
- reports and portable evidence

Proof:

- All 44 evaluated consequential paths met the Code Capability criteria.
- 1,322 supporting Action Path evidence traces.
- 555 Verified Action Frontier items record how far each path is currently verified and what evidence would strengthen assurance.
- Evidence pack is hash-bound and independently verifiable.
- Public report, System Consequence Map, executive / technical / auditor outputs, machine JSON, Passport, and Decision Receipt project from the same evaluated evidence base.

3. INNOVATION & ORIGINALITY

Judge question:

Is this meaningfully different from expected implementations?

Our answer:

HAIEC shifts the assurance unit from “tool,” “permission,” or “finding” to the complete AI Action Path Trail and the consequence it reaches.

Differentiators:

- Tool inventory says what exists. HAIEC establishes qualified continuity to consequence.
- IAM says what an identity may be permitted to access. HAIEC compares actual code capability with requested, policy-authorized, effectively granted, and observed evidence.
- Source scanners can find code patterns. HAIEC composes qualified source-backed relationships into consequential action paths, assurance evidence, and release decisions.
- Runtime monitoring shows what happened. HAIEC can establish what the source makes reachable before runtime and keep Observed separate.
- Consequence Delta compares what changed in the consequence surface between qualified evaluations.
- Portable assurance artifacts let evidence travel with the evaluated release.

Originality proof:

The public ORIGINALITY.md and Git history separate the pre-existing HAIEC foundation from the MunichTech-period implementation.

4. PRACTICAL APPLICABILITY & SCALABILITY

Judge question:

Can enterprises actually use this?

Our answer:

Yes. HAIEC operates as a SaaS and supports both standard capability recognition and enterprise-specific capability extensions.

Standard-first:

Common actions are recognized through HAIEC’s capability ontology and qualified analysis. Customers do not need to define every action before an evaluation can run.

Enterprise extension:

Proprietary actions can be added when required, for example:

- Telecom: `restartCell()`, `changeAntennaTilt()`, `publishRAppPolicy()`
- Banking: `releaseWireTransfer()`, `freezeAccount()`, `adjustCreditLimit()`
- Manufacturing: `restartProductionCell()`, `changeRobotSpeed()`, `publishPLCConfiguration()`

Enterprise workflow:

Source handling: GitHub-authorized access → transient analysis → source workspace discarded → structured evidence retained. For stricter environments, a customer-controlled private analyzer can be scoped as a PoC deployment pattern.

Evaluate source

- inspect actual capability
- define or import the approved Operating Envelope
- compare capability with intended authority
- resolve review flags
- release with a portable evidence record
- compare the next release with Consequence Delta.

Potential users:

AI platform engineering, AppSec, cloud/security architecture, AI assurance/governance, release management, internal audit, and independent assurance teams.

5. PRESENTATION & COMMUNICATION

Judge question:

Can the team explain and demonstrate the value clearly?

Recommended 90-second judge path:

1. Open the KestrelVoice public report.
2. See REVIEW and 44 consequential paths.
3. Open the System Consequence Map.
4. Select ORDER RECORD WRITE.
5. Inspect the AI Action Path Trail.
6. See the five independent evidence planes.
7. See the Verified Action Frontier.
8. Verify the evidence pack with one command.

Judge materials:

- Judge Deck — Final Submission: concise pitch and judge narrative.
- Technical Evaluation Companion — Final Submission: deep technical defense.
- 2–3 minute demo video: live product proof.
- Technical Judge FAQ: objection handling.
- Evidence pack and verifier: independent verification.

ALIGNMENT WITH “BUILD WHAT EUROPE NEEDS”

MunichTech emphasizes deployable solutions, European competitiveness, enterprise software, Industry 4.0, applied AI, and digital sovereignty.

HAIEC contributes:

- provider-independent assurance for consequential AI;
- inspectable source-backed evidence;
- enterprise-defined operating boundaries;
- technical accountability without relying on opaque AI judgment;
- applicability to telecom, manufacturing, finance, infrastructure, public services, and enterprise automation.

CLOSING SCORECARD

Problem relevance: real and increasingly urgent.

Technical excellence: live, deterministic, source-backed, independently verifiable.

Innovation: consequence-centered assurance rather than inventory-only or permission-only views.

Practicality: SaaS, enterprise extensibility, release workflow, portable evidence.

Presentation: live public demo, minimal deck, deep technical deck, verifier, and guided judge path.

The question:

What can your AI actually cause

EXTERNAL-JUDGE QUESTION COVERAGE ADDED

Automation question: covered by the deterministic source-backed path story. Public answer should say HAIEC automatically identifies and qualifies supported AI-facing action paths from the evaluated source snapshot, while enterprise inputs are optional for proprietary semantics and operating boundaries.

Completeness question: covered by analysis-coverage discipline. Do not claim perfect repository coverage. Claim bounded, inspectable evidence and separate unresolved analysis surfaces from established paths.

Verifier question: covered by evidence integrity. The verifier validates artifact integrity and consistency of the evidence package; it is not represented as an independent full semantic re-analysis unless that capability is later added.

Security/source question: covered by source-code trust model. Source is processed transiently through authorized GitHub access and is not retained as a persisted repository copy. Structured evidence, hashes and assurance outputs are retained.

Enterprise workflow question: covered by Monday-morning flow. Connect source → evaluate standard capabilities → inspect map → add proprietary capability packs where needed → define Operating Envelope → resolve release flags → re-evaluate → carry evidence → compare next release.

Business value question: frame around faster release review, better consequence-change visibility and reusable assurance evidence across engineering, security, governance, release and audit.

Do not add unsupported numbers for accuracy, scan runtime, maximum repo size or ROI until measured.

?